HacdsDeFi Teknik Raporu

2026-10-21 Sürüm 2

Başlamadan önce

hacds, Hacash ana ağında bir zincir üstü varlıktır, serial 2000. Takas, piyasa yapıcılığı, stake, kredi ve hazine işlevlerinin tamamı tek bir HVM kontratında çalışır. Oracle, indeksleyici ve cüzdan da tarafımızca yazıldı ve kendi sunucularımızda çalışıyor.

Hacash'in ilk DeFi kontratı üzerinde çalışıyor. Hacash, HVM'yi (Turing-tam akıllı kontrat VM'i) 765.432 blok yüksekliğinde etkinleştirdi ve bu kontrat, dört DeFi ilkelini — takas ve piyasa yapıcılığı, stake, kredi, hazine — tek bir kontratta toplayan ana ağdaki ilk uygulamadır. Kendi dağıtım listemizi de "ana ağdaki ilk Turing-tam akıllı kontrat" başlığıyla yazmıştık. Oracle, indeksleyici ve cüzdan da bize ait ve kendi sunucularımızda çalışıyor.

Bu belge dört bölümden oluşur: önceki sürümde neyin yanlış gittiği, bu sürümün nasıl tasarlandığı, dört özelliği ve mevcut durum.

Önceki "Hacds Yazıt Teknik Raporu" geçersizdir. O tasarımı yaptık, yayına aldık ve kapattık.

---

1. Önceki sürüm

1.1 Neydi

Önceki sürüm hacds'i HACD elmaslarına kazınan bir yazıt olarak tanımlıyordu. Dört parçadan oluşuyordu.

Yazıtla arz. Her HACD bir Hacds yazıtı taşır ve elmas başına yalnızca biri geçerlidir. Sıradan bir elmas 10.000 hacds değerindeydi; özel bir elmasın ne kadar edeceği (kalp için 500.000, dört ardışık harf için 300.000, özel şekil için 200.000) topluluk oylamasıyla belirleniyordu.

Eklentiyle ticaret. Merkeziyetsiz borsa MoneyNex adlı bir eklenti cüzdanı olarak yapıldı. Kullanıcılar eklentiyi kurup onun cüzdanı üzerinden işlem yapıyordu.

Hedef olarak listelenme. Teknik raporda yazılan ilk listeleme hedefi xeggex.com idi.

Zincir taramasıyla keşif. Her istemci tüm zinciri kendisi tarayıp yazıtları arıyordu; üçüncü taraf sunucular ek olarak arama sağlayabiliyordu.

1.2 Yaptık ve listelendi

Bunların hiçbiri kâğıt üstünde bir tasarım değildi. Yazıt sistemi yapıldı, MoneyNex eklentisi çalıştı ve hacds iki borsada listelendi: xeggex.com ve nonkyc.io.

Sonra işlem durduruldu.

1.3 Nerede yanlış gitti

Kök neden ilk maddedir. Diğer üçü onun sonucudur.

Elmasa kazınan bir para hareket edemez. 10.000 hacds tutmak, belirli bir HACD elması setini tutmak demekti ve transfer, elmasların da birlikte taşınmasını gerektiriyordu. Bir elmas tek bir yazıt taşır, dolayısıyla toplam arz elmas sayısına sabitlenmişti — bölünemez ve parçalanamaz. Parçalanamayan bir şey para olamaz ve borsalar üzerinde normal yatırma, çekme veya derinlik yürütemez.

Fiyatın çıpası yoktu. Arz oranları oylamadan geliyordu, yani her elmas sınıfının kaç hacds ettiği oylamayla her an değiştirilebiliyordu. Borsaların yeniden üretilebilir bir dönüşüm dayanağı yoktu, piyasa yapıcıların istikrarlı kotasyonu olamazdı ve derinlik hiç oluşmadı.

Ticaret başkalarına bağlıydı. Merkeziyetsiz borsa bir üçüncü taraf eklentisiydi — eklenti güncellenmezse borsa durur. Merkezi borsalar daha doğrudan: ticaret ve dolaşım tamamen xeggex ve nonkyc'in elindeydi.

Yedek yol yoktu. Bu iki borsa tüm dolaşım kanalıydı. Durduklarında zincir üstünde işlem yapılacak başka yer kalmıyordu.

Topluca: para serbestçe dolaşamadığı için fiyatlanamıyordu; fiyat olmadan piyasa yapıcılığı yapılamıyordu; piyasa yapıcılığı olmadan tamamen borsalara bağımlıydı; borsalar durduğunda her şey durdu.

Listelenmek hiçbir zaman asıl mesele değildi. Listelenme tasarım hatasını ortadan kaldırmaz — yalnızca işlemin durdurulduğu güne erteler. Bu sürümün tasarımı değiştirmesinin nedeni budur.

1.4 Ne değişti

Tek bir şey: para elmaslardan ayrıldı.

hacds, hiçbir elmasa bağlı olmayan, serbestçe bölünebilen ve transfer edilebilen standart bir zincir üstü varlık oldu. Finansal mantık eklentilerden ve borsalardan kontrata taşındı. Dolaşım artık listelenmeye bağlı değil — zincir üstü AMM kendi başına işlem yapıyor ve listelenme, hayatta kalmanın ön koşulu değil isteğe bağlı bir seçenek haline geliyor.

---

2. Bu sürümün tasarımı

2.1 Arz sayısı nereden geliyor

hacds arzı havadan seçilmedi. Zincirdeki her elmastan hesaplandı.

Her HACD, HIP-5'in beş boyutunda puanlanır (şekil, stil, renk, isim, numara). Bir puan bir hacds'e dönüşür (kontratta `SCORE_SCALE = 1e8`, yani 1 puan = 1e8 atoms). Toplam, tüm elmasların tam ortalama puanıdır:

```
ortalama puan × toplam HACD arzı
= 3.55440306 × 16.777.216
= 59.632.987 hacds
= 5.963.298.700.000.000 atoms
```

Bunu herkes yeniden hesaplayabilir: tarayıcı API'sini çekin, her elması puanlayın, aynı sayıyı bulursunuz. Önceki sürümde eksik olan tam olarak buydu — kimsenin kararına bağlı olmayan bir dayanak.

Puanlama zincir dışında hesaplanır ve imzalanır; kontrat, sertifikayı kabul etmeden önce puanı zincir üstünde yeniden hesaplar. Zincir dışı gaz tasarrufu sağlar, zincir üstü doğruluğu korur ve iki taraf da aynı algoritmayı kullanır, dolayısıyla sonuçlar aynı olmak zorundadır. Bunu 600'den fazla gerçek elmas üzerinde doğruladık.

2.2 Varlık özellikleri

Öğe	Değer
serial	2000
ticket / name	HACDS / hacds
decimal	8
toplam arz	5.963.298.700.000.000 atoms
issuer	VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1 (kontrat)
arz maliyeti	8 HAC (protokol ücreti, blok ödülüne göre değişir)

Not: zincirdeki serial #1028'in ticket'ı da HACDS'tir (adı "HACD Fragments" ve başkasına aittir). Protokol, isim veya ticket benzersizliğini kontrol etmez — yalnızca serial'in alınıp alınmadığına bakar. serial 2000'i esas alın.

---

3. Dört özellik

3.1 Açık kaynak

Kontrat (fitsh kaynağı), oracle, indeksleyici ve cüzdanın tamamı yayımlanacak.

Bir DeFi protokolü başkalarının parasını çalıştırır ve kullanıcıların kendini korumasının tek yolu kodu okuyabilmek ve hesabı kontrol edebilmektir. Önceki cüzdan bir üçüncü taraf eklentisiydi, kullanıcı ya güvenmek ya da güvenmemek durumundaydı. Bu sürüm mantığı açığa koyuyor.

Kod açıldığında iki şey doğrudan mümkün olur:

- **Puanlama bağımsız olarak yeniden hesaplanabilir.** Herkes zincirdeki tüm elmasları çekip aynı algoritmayla yeniden puanlayabilir ve arz sayısının doğru olup olmadığını görebilir. Bunu 600'den fazla elmas üzerinde kendimiz yaptık.
- **Kontrat davranışı denetlenebilir.** Yöneticinin ne yapıp ne yapamayacağı fonksiyon listesinde yazılı — bize inanmanıza gerek yok.

3.2 Merkeziyetsizlik

Bunun sınırlarını belirtmek gerekir. Niteliksiz "tamamen merkeziyetsiz" demek anlamsızdır.

Hâlihazırda geçerli olanlar:

Kurallar ve yürütme zincir üstünde. AMM fiyatlaması, kredinin geçme koşulları, tasfiye eşiği, komisyon dağılımı — hepsi kontrat kodudur. Katılımcılar aynı kurallarla işlem yapar; biz ortada değiliz ve araya giremeyiz.

Kontrat yükseltilemez ve arka kapısı yoktur. Kendini yok etme fonksiyonu, yükseltme fonksiyonu ve tek bir kullanıcıyı donduracak fonksiyon yoktur. Dağıtılan neyse o kalır.

Yönetici kullanıcı fonlarını elliyemez. Bu, fonksiyon listesinden doğrudan doğrulanabilir: yöneticinin çağırabildiği altı fonksiyonda çekim yok, bakiye değişikliği yok, teminat değişikliği yok, borç değişikliği yok ve herhangi bir varlığı taşıma yetkisi yok. Yalnızca parametreleri değiştirir.

Yönetici yetkilerinin katı sınırları var. Sözle değil, koddaki kontrollerle:

İşlem	Sınır
`set_fee` — AMM komisyon oranı	≤ 300 bps (%3)
`set_loan_rate` — kredi faiz oranı	≤ 2000 bps (%20)
`set_liquidate_bonus` — tasfiye primi	≤ 1000 bps (%10)

Teminat oranı bir parametre değildir. `COLL_RATIO` bir kontrat sabitidir, 2'ye sabitlenmiştir. Yönetici değiştiremez, dolayısıyla "teminat oranı gizlice düşürülebilir mi" sorusu yoktur.

Yönetici değiştirilemez. Dağıtımda `init_admin` ile sabitlenir ve `set_admin` yoktur. Adres değiştirilemez.

Katılım izin gerektirmez. Takas, piyasa yapıcılığı, stake ve kredi herkese açıktır — beyaz liste yok, onay yok, bizim rızamıza gerek yok. Hazine adresi de sabit bir sabittir.

Varlıklar sizin elinizde kalır. hacds, kullanıcının kendi adresinde tutulan standart bir zincir üstü varlıktır. Biz saklamıyoruz ve donduramıyoruz.

Henüz geçerli olmayanlar, açıkça:

Oracle şu anda tek noktadır ve bizim tarafımızdan çalıştırılır. Varlıklarınızın değerini belirleyemez — kontrat puanı zincir üstünde yeniden hesaplar — ama takasın çalışıp çalışmayacağını belirler. Düşerse takas durur.

Yöneticinin elinde `set_paused` var, kontratı genel olarak duraklatabilir. Varlıklarınızı durduramaz ve kimsenin parasını alamaz, ancak protokolün geçici olarak işlem kabul etmemesini sağlayabilir.

Bu ikisi bugünkü merkeziyetsizliğin gerçek sınırlarıdır. Bunları kaldırmak kontrat değişikliği gerektirir (birden fazla oracle, zaman kilidi, multisig) ve bir sonraki kontrat güncellemesiyle gelecektir.

3.3 Güvenlik

İki taraf: kullanıcı fonlarının nasıl tutulduğu ve protokolün kurallarının nasıl güvence altına alındığı.

Özel anahtar ağa hiç çıkmaz. Cüzdan soğuk ve sıcak tarafa ayrılmıştır. Çevrimdışı makine işlemi oluşturur ve imzalar; çevrimiçi makine yalnızca sorgular ve yayınlar. Özel anahtar tam olarak bir kez,

ana süreç belleğinde görünür ve alt sürece ortam değişkeniyle verilir — diske hiç yazılmaz. Bu, Hacash işlemlerinde nonce ve zaman aşımı olmadığı için mümkündür: çevrimdışı imzalanan bir işlem geçerliliğini korur ve çok sonra yayınlanabilir.

İmzalama yerelde yapılır. İşlemler paketle gelen Rust programı tarafından yerel makinede imzalanır, hiçbir düğüm üzerinden geçmez. Düğüm yalnızca çeviri, doğrulama ve yayın için kullanılır. Düğümün `/util/transaction/sign` uç noktası özel anahtarın HTTP üzerinden gönderilmesini gerektirir — kullanmıyoruz.

Cüzdan dosyaları şifrelidir. Parola scrypt ile türetilir ($N=2^{15}$, $r=8$, $p=1$); şifreleme AES-256-GCM, biçim sürümü AAD olarak bağlanır.

Arayüz sürecinin sistem erişimi yoktur. Renderer `contextIsolation` açık ve `nodeIntegration` kapalı çalışır ve ana süreci yalnızca preload beyaz listesi üzerinden çağırabilir. Renderer'a kötü niyetli içerik girse bile dosya sistemine veya özel anahtara erişemez.

Oracle'a körü körüne güvenilmez. İmzalı sertifika yalnızca bir taşıma belgesidir. Kontrat aldığında puanı kendi motoruyla zincir üstünde yeniden hesaplar ve karşılaştırır. Ele geçirilmiş bir oracle bile puan uyduramaz.

Varlıklar korunur. Zincir üstü özdeşlik: kullanıcı varlıkları artı kontrat varlıkları toplam arza eşittir. Her işlem bunu korur ve herkes her an kontrol edebilir.

Koruma önlemleri var. Kova başına gaz sınırı, işlem başına yalnızca bir elmas yatırılabileceği anlamına gelir; cüzdan bunu sert biçimde engeller ve çoklu seçimi otomatik böler. Toplu işlem tamamlandıktan sonra ters buton beklemeye alınır, böylece geri alımdan hemen sonra yanlışlıkla tekrar yatırmazsınız.

3.4 Eksiksiz DeFi

Önceki sürümde iki finansal işlev vardı: para çıkarmak ve borsaya sokmak. İlkel yoktu — piyasa yapıcılığı yok, teminat yok, borç yok ve fiyat artışı dışında getiri yok.

Bu sürüm bunu tamamlıyor. Tek kontratta dört parça:

Modül	Ne yapar
AMM	HAC ile hacds arasında iki yönlü takas. Sabit çarpım `x × y = k`, %0,5 komisyon: LP %50 / stake havuzu %30 / hazine %20
Stake	LP payları komisyon kazanmak için stake havuzuna kilitlenir, rpu sabit noktalı dağıtım, tembel kapanış — alınmayan ödül kaybolmaz
Kredi	hacds teminatıyla HAC borç alınır. %50 teminat oranı, yıllık %5, geri ödenebilir, oran çok düşerse tasfiye edilir
Hazine	Komisyonlar burada birikir ve düzenli olarak süpürülür

İki araç daha gelir.

Atomik işlemler. Birden fazla işlem tek bir işlemde birleşebilir — hepsi başarılı ya da hepsi başarısız — protokol sınırı 200 eylemdir. "Elmas yatır + hemen hacds al" tek işlem olabilir.

Tasfiye. Kredinin tanımlı bir tasfiye çizgisi ve primi vardır. %200 teminat oranının altına düşen bir pozisyonu gören herkes tasfiye edebilir; bizim dahil olmamıza gerek yok. İnsanların tasfiye edilebilir hesapları bulabilmesi için indeksleyiciyi yazdık (bkz. 4.3), çünkü kontrat borçluları listeleyemez.

Faiz tembel hesaplanır: zamanlanmış görev yok, keeper bağımlılığı yok. Biri kontratla etkileşime girdiğinde faiz, blok yüksekliği farkına göre tamamlanır. Gaz tasarrufu sağlar ve kimse bakım yapmasa bile çalışmaya devam eder.

---

4. Sistem bileşimi

Dört katman: zincir üstündeki kontrat ve varlıklar, sunucu tarafındaki düğüm, oracle ve indeksleyici, istemcideki cüzdan ve kullanım biçimi olarak soğuk/sıcak ayrımı.



`GET /cert?name=X` (:8123)	HACD puanlama sertifikası	126 B; yatırma/geri alma için gerekli
`sandboxcall` (:8082)	Kontrat durumunu oku	IP başına eşzamanlılık ≤ 2; sırala
`GET /watchlist` (:8124)	Borçlu defteri + sağlık	Tek anlık görüntü; yerel taramayı gereksiz kılar
`POST /submit/transaction`	Yayınla	`ret:0` yalnızca havuza girdiğini gösterir; blok 300 sn

Tek doğruluk kaynağı: `wallet-app/设计文档.md §0` | Birimler: HAC için HAC, hacds için her yerde ham atoms (÷1e8 dönüşümü yok) | VM kova sınırları: compute 18.009 / resource 6.100 / storage 111.911

4.1 Kontrat

Kontrat, Hacash VM üzerinde fitsh ile yazılmıştır. Dört modülün kuralları yukarıda anlatıldı; burada birkaç uygulama tercihi var.

Teminat, elmasların kendisi değil, elmasların kontrata yatırılmasıyla doğan hak olan `due` kotasıdır. Kullanıcılar borç almak için elmaslarını kilitlemek zorunda değildir.

Teminat doğrudan AMM'nin canlı havuz fiyatıyla değerlenir (`coll_to_hac = coll × amm:hac / amm:ds`). Dış fiyat beslemesi kullanılmaz; bu, bütün bir oracle altyapısını ortadan kaldırır ve fiyat beslemesi saldırılarına açık yüzeyi küçültür.

Hazine adresi kontratta sabittir ve değiştirilemez.

Kontrat yapılandırmayı okumak için bir uç nokta sunmaz, dolayısıyla yönetici arayüzü mevcut komisyon veya faiz oranını gösteremez. Bu bilinçlidir: yalnızca değerleri yansıtmak için çalışan bir kontratı yükseltmeye değmez.

Önceden bilinmesi gereken bir katı sınır var. Hacash VM gazı, toplam bütçe `gas_max` ile ilgisiz üç bağımsız kovaya ayrılır:

Kova	Sınır
compute	18.009
resource	6.100
storage	111.911

resource satırı, **işlem başına en fazla bir HACD yatırılabilmesini** zorunlu kılar. Bir elmasın sertifikasını doğrulamak puanın zincir üstünde yeniden hesaplanmasını gerektirir ve yaklaşık 3.000 resource tüketir; iki elmas 6.101 eder, 6.100'ün hemen üstünde. Bunun `gas_max` veya komisyonla ilgisi yoktur — onları yükseltmek işe yaramaz, yalnızca işlemi bölmek yarar. Bu bir konsensüs parametresidir; değiştirmek çatallanma demektir.

4.2 Oracle

Kontrat bir elmasın ne kadar değerli olduğunu doğrulamak zorundadır, ancak puanlama SHA3 ve beş boyutlu bir değerlendirme içerir ve zincir üstünde tamamen çalıştırmak çok pahalıdır. Çözüm: zincir dışında hesapla, zincir üstünde doğrula.

`GET /cert?name=KWTVSH` çağrılır. Sunucu elmasın genlerini alır, `visual_gene` türetir, beş boyutlu puanı hesaplar ve 126 baytlık bir sertifikaya (29 bayt mesaj + 97 bayt imza) paketleyip kendi özel anahtarıyla imzalar. Kontrat puanı zincir üstünde yeniden hesaplar ve karşılaştırır; imza sahteciliği önlemek içindir.

Oracle 43.251.101.101:8123 üzerinde çalışır ve yalnızca Python standart kütüphanesine bağlıdır (`hashlib.sha3_256` yerleşiktir). Özel anahtar sunucuda kalır; istemciler yalnızca sertifika alır.

Kontrattaki `cfg:oracle` tam olarak bir sabit adres tanır. Kendi çalıştırdığınız bir oracle tarafından imzalanan sertifika kabul edilmez — bu bilinçlidir, aksi halde puanlar istenildiği gibi uydurulabilirdi.

4.3 İndeksleyici

Burada etrafından dolaşmanın mümkün olmadığı bir sorunla karşılaşıyoruz: kontrat borçluları listeleyemez. Kredi kayıtları adresten oluşturulan anahtarlarda tutulur (`loan:debt:<adres>`); kontratta liste yok, günlük yazılmaz ve düğüm kontrat KV'sini önekle listeleyemez. Yani "şu anda kim tasfiye edilebilir" sorusunun zincir üstünde cevabı yoktur.

Çözüm, zinciri çevrimdışı tarayarak kurulan bir defterdir: blokları gez, kredi fonksiyonu seçicisine çarpan kontrat çağrılarını ayıkla, işlemin main adresini borçlu olarak al, `lastScanned`'i artımlı kaydet ve defteri diske yaz. Anlık görüntü için defterdeki her adres için `loan_status` sorgulanır.

Cüzdanın ihtiyaç duyduğu tek uç nokta: `GET /watchlist`, özeti ve sağlık değerleriyle birlikte defteri döner. Diğerleri — `/snapshot`, `/health`, `/sync`, `/watch/add`, `/watch/remove` — yardımcıdır.

İndeksleyici sunucu tarafında olmak zorundadır. Ölçülen tarama hızı sıralı olarak 2,6 blok/sn, eşzamanlılık 4'te 6,6 blok/sn; 500 blok geriye gitmek 15 ile 75 saniye ve binden fazla HTTP isteği demektir. Bir telefon bunu veri ve pil açısından karşılayamaz. Sunucu tarafında olduğunda cüzdan tek istek yapar. İki yan fayda daha: Python standart kütüphanesinde SHA3 vardır (Electron'un Node çalışma zamanında yoktur, aşağıdaki cüzdan bölümüne bakın) ve tek bir defter vardır, yani PC ile mobil aynı şeyi görür.

4.4 Cüzdan

Cüzdan bir Electron uygulamasıdır, soğuk/sıcak birleşik: çevrimdışı makine oluşturur ve imzalar, çevrimiçi makine sorgular ve yayınlar.

Arayüz, çerçevesiz sade HTML/CSS/JS'tir. Varlık özeti, protokol durumu, kalıcı kredi durumu kartı, canlı AMM fiyatları, HACD galerisi, çevrimdışı imzalama, atomik işlemler, tasfiye konsolu ve yönetici sayfası içerir. Metinler Çince, İngilizce ve Türkçe olarak yerelleştirilmiştir, 479 kayıt.

Ana süreç IPC beyaz listesini, cüzdan keystore'unu, yinelemeli ücret tahminini ve üç servise HTTP'yi yönetir.

İşlem oluşturma ve imzalama, paketle gelen Rust programı `hacds_txbuild`'e devredilir; sekiz modu vardır. Yeniden yazmadık çünkü mantık karmaşıktır (200'den fazla eylem, Type-3 tel formatı, ücret yinelemesi), yeniden yazmak risklidir ve bu kod zaten çalışıyor.

Cüzdanda bahsetmeye değer birkaç şey:

- Kredi sayfası, giriş kutularının altında kullanılabilir borç limitini ve bu kredinin gerektirdiği teminatı canlı gösterir; ana ekranda kalıcı bir kredi durumu kartı da vardır. Önceden

yalnızca "borç al" düğmesine basan bir kullanıcı, açıklama olmadan kontratın `return 40` yanıtını alıyordu.

- Çoklu HACD seçip toplu işlem yapıldığında, işlem başına bir elmas olacak şekilde otomatik bölünür.
- Çevrimdışı imzalama: soğuk tarafta oluştur ve imzala, parçalı QR veya dosya üret, sıcak tarafta okut ve yayınla.
- Atomik işlemler: tek işlemde birden fazla eylem.

---

5. Eski ve yeni

	Önceki	**Şimdi**
Token biçimi	HACD üzerinde yazıt	Standart zincir üstü varlık (serial 2000)
Arz	Elmas sayısıyla sınırlı, elmas başına bir yazıt	59.632.987 olarak bir kez arz edildi, serbestçe bölünebilir
Değer kaynağı	Topluluk oylaması	Elmas puanlama algoritması, yeniden hesaplanabilir
İşlem yeri	MoneyNex eklenti cüzdanı	Zincir üstü AMM kontratı
Listelenme	xeggex.com ve nonkyc.io'da listelendi, sonra durduruldu	Listelenmeden işlem görür; standart varlık arayüzüyle listelenebilir
Finansal ilkeller	Yok	AMM / stake / kredi / hazine
Puan kaynağı	İstemci tarafı hesaplama	Oracle imzalı sertifika + zincir üstü yeniden hesaplama
Veri keşfi	Her istemci zinciri yazıtlar için tarar	Sunucu tarafı indeksleyici, cüzdandan tek çağrı
Cüzdan	Üçüncü taraf eklenti	Kendi Electron konsolumuz
Özel anahtar	Eklentiye verilir	Yerelde tutulur, ağa hiç çıkmaz
Kaynak kodu	Kapalı	Tamamen açılması planlanıyor
Kuralları kim değiştirebilir	Eklenti yazarı, borsalar	Yalnızca zincir üstü kontrat; yönetici sadece sınırlı parametreleri değiştirebilir

---

6. Durum

Yayında olanlar:

Öğe	**Nerede**
Kontrat	VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
hacds varlığı	serial 2000, arz 5.963.298.700.000.000 atoms

Öğe	Nerede
Tam düğüm	43.251.101.101:8082
Oracle	43.251.101.101:8123
İndeksleyici	43.251.101.101:8124
Cüzdan	HacdsDeFi-Setup-0.1.0-x64.exe / HacdsDeFi-0.1.0-x64.exe

Zincirde bir korunum özdeşliği var: kullanıcı varlıkları artı kontrat varlıkları toplam arza eşittir. Kullanıcıların elindeki, kontrattan geri alınmış olan kısımdır.

Doğruladıklarımız:

- Puanlama oracle ile 152 elmasta ve kontratın zincir üstü motoruyla 610 elmasta uyuşuyor, hepsi eşleşiyor
- Tek elmas yatırma başarıyla yayınlanıyor
- İki elmas yatırma `OutOfGas(31): resource ... used 6101 > limit 6100` bildiriyor, beklendiği gibi
- Gerçek makinede 3 elmas toplu yatırma: 3/3 başarılı; aynı zaman damgasıyla oluşturulan işlemlerin hash'leri farklı
- Paketlenmiş öz test `RESULT = ALL OK`, sekiz motor modunun tamamı mevcut
- Üç dilin her birinde 479 kayıt, anahtar kümeleri eşleşiyor

---

7. Bilinen sınırlamalar

Bazıları eksiklik değil, şimdilik kabul ettiğimiz ödünleşimlerdir. Sonraki kişi aynı taşlara basmasın diye yazılmıştır.

Oracle tek noktadır. Onu biz çalıştırıyoruz, adresi kontratta sabittir ve değiştirilemez. Dağıtılmış puanlama yerine güvenilir puanlamayı seçtik: puanlar nesnel ve yeniden hesaplanabilir, kontrat da zincir üstünde yeniden hesaplayarak arka destek sağlar — ancak makinenin çevrimiçi kalması gerekir.

Yönetici kontratı duraklatabilir. Fonları taşıyamaz, ancak protokolün geçici olarak işlem kabul etmemesini sağlayabilir. Parametre kontrolü de sayısal sınırlarla birlikte yöneticide toplanmıştır.

Yatırma işlem başına bir elmasla sınırlıdır. Nedeni yukarıdaki gaz bölümünde. Daha büyük tutarların birden fazla işleme bölünmesi gerekir ve arayüz bunu söyler.

300 saniyelik bloklar. Başarılı yayın, zincire girdiği anlamına gelmez; `ret:0` yalnızca havuza girdiğini gösterir. Cüzdan blok ritmine göre birkaç kez tazeler (15, 45, 90, 180, 320 saniye).

Faiz ayrıca gösterilmez. Kontrat tembel hesaplar ve tahakkuk başlangıcı dışa açılmaz, dolayısıyla o an ödenecek faiz hesaplanamaz. Yalnızca geri ödeme sırasında zincir üstü kapanış sonucu olarak görünür.

İlk ikisi merkeziyetsizliğin sınırlarıdır; diğer üçü kontratın hâlihazırda yayında olmasından gelir. Bunları değiştirmek kontratı yeniden dağıtmak ve varlığı yeniden arz etmek anlamına gelir; bu,

kazancından çok daha pahalıya gelir, o yüzden şimdilik kalıyor. Yazmak gizlemekten iyidir.

---

8. Sırada

Android istemcisi geliştiriliyor — Kotlin ve Compose, işlem baytları eşleşsin diye aynı Rust motorunu kullanıyor. aarch64 çapraz derleme çalışıyor.

Merkeziyetsizlikte sıradaki adım, önceki bölümün ilk iki maddesini kaldırmak: birden fazla oracle örneği ve kilit parametrelerde zaman kilidi. İkisi de kontrat değişikliği gerektirir ve bir sonraki kontrat güncellemesiyle gelecektir.

Sırada bekleyenler: gönderim öncesi parametre doğrulama, her bloktan sonra otomatik bakiye tazeleme, işlem geçmişi, toplu işlemler.

Peşinde olmadığımız yönler: yerelde çalıştırılan oracle (kontrat kabul etmez), çok düğümlü yayın, otomatik güncelleme.

Ek A: Adresler ve parametreler

Öğe	Değer
Dağıtıcı / yönetici	15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q
Kontrat	VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
Hazine	1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK
Oracle hesabı	1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu
hacds serial	2000 (decimal 8, ticket HACDS)
SCORE_SCALE	1e8 (1 puan = 1 hacds)
COLL_RATIO	2 (%50 teminat oranı, tasfiye çizgisi 200, kontrat sabiti, değiştirilemez)
AMM komisyonu	%0,5, LP %50 / stake %30 / hazine %20
Kredi oranı / tasfiye primi	yıllık %5 / %2
Yönetici parametre sınırları	komisyon %3 / oran %20 / tasfiye primi %10
Sertifika uzunluğu	126 bayt (mesaj 29 + imza 97)
VM kova sınırları	compute 18.009 / resource 6.100 / storage 111.911
Blok hedefi	300 saniye

Ek B: Birimler

hacds, decimal 8 bir varlıktır. Zincirdeki tam sayı atoms cinsindendir ve bir arayüz yazarken birim ile adet karıştırmak çok kolaydır.

Cüzdanın yaklaşımı basit: hacds her zaman atoms olarak gösterilir, ek bir dönüşüm katmanı yoktur. Bakiyeler, havuzlar, teminat, giriş kutuları, fiyatlar — hepsi atoms cinsinden, tarayıcı ve borsa arayüzleriyle aynı. Giriş etiketleri atoms yazar ve varsayılan değer 100000000'dir, böylece kullanıcı bir hacds'in kaç atoms olduğunu doğrudan görür.

Adede çevirmek için kendiniz 1e8'e bölün. 7000000000 atoms, 70 hacds'tir.

Üç kural:

1. Zincirde, borsa arayüzlerinde ve cüzdanda her zaman atoms kullanın, değiştirmeden. Adet isteyen tüketici 1e8'e kendi böler.
2. 1e8'e bölünmüş değeri görüp giriş kutusuna onu yazmayın. Kullanıcı 70 görüp 70 yazarsa, 70 atoms göndermiş olur — 1e8 kat sapma. Giriş etiketleri atoms yazmalıdır.
3. Toplam arz 5.963.298.700.000.000, JS `Number.MAX_SAFE_INTEGER` değerinin %66 üzerindedir. JS'te BigInt, Kotlin'de BigInteger kullanın; `Long` yetmez.

HAC farklı çalışır: zincir üstü birim zhu'dur (1 HAC = 1e8 zhu) ve arayüz her zaman HAC kullanır. Dolayısıyla `add_liquidity(hac_amt, ds_amt)` fonksiyonunun iki parametresi farklı birimlerdedir — biri zhu, diğeri atoms. 1e8 hatası yapmanın en kolay yeri burasıdır; ilk likidite enjeksiyonunda biz de yaptık.